MOSAIC – PS 2

Detailed Report of the Solution

Multimodal AI for Geometry and Mathematics Problem Solving

Abstract. This report documents the design and implementation of a custom multimodal AI system built to solve multiple-choice geometrical and mathematical problems presented as image–text pairs. The pipeline processes geometric diagrams using a fine-tuned EfficientNet-B4 and EasyOCR, encodes natural-language problem descriptions using BERT, fuses both modalities through a multi-head attention fusion layer, decodes the joint representation via a T5-small decoder under teacher forcing, and performs final answer selection using the Phi-1.5 lightweight language model. The entire system is trained end-to-end via backpropagation with a hybrid loss function, achieving a maximum test accuracy of 29.83% and a validation accuracy of 33.42% on the competition-provided dataset under strict single-GPU resource constraints.

Hardware: NVIDIA T4 GPU (Google Colab)
Dataset: Competition-provided geometry MCQ dataset (100% utilised)
Domain: Computer Vision · NLP · Multimodal Deep Learning

Contents

1 Problem Statement

The problem statement revolves around the creation of a multimodal AI model. The task involves a dataset containing multiple-choice geometrical and mathematical problems in the form of images, along with a problem description. These problems are first to be analysed using techniques like computer vision and natural language processing to interpret their semantic and wholesome context. Thereafter they are to be fed to a deep learning and reasoning model to finally reason, apply logic, and predict an answer out of the four available options.

2 Inspiration from Existing Models and Frameworks

2.1 Inter-GPS: Interpretable Geometry Problem Solver

Inter-GPS is a novel geometry problem solving approach with the following defining characteristics:

- It parses the problem text and diagram into formal language automatically via rule-based text parsing and neural object detection, respectively.
- Unlike implicit learning in existing methods, Inter-GPS incorporates theorem knowledge as conditional rules and performs symbolic reasoning step by step.

This is the canonical approach to solve geometry problems in the Geometry3K dataset.

2.2 Why Inter-GPS Cannot Be Incorporated Currently

As per the rules of the problem statement, only the dataset provided may be used for all training, analysis, and preprocessing tasks. The highlight of Inter-GPS is step-by-step symbolic reasoning based on rule-based conditionals and theorems, along with implicit learning. Implementing this would naturally require either:

- A database of geometry problems following a heuristic approach, or
- A RAG-based knowledge graph creation, by which the reasoning model retrieves relevant information and dynamically includes it in its chain of thought to simulate a step-by-step symbolic reasoning process.

> This approach mimics the way humans actually solve math problems — understanding problem context, then applying step-by-step symbolic and theorem-based reasoning to arrive at the final answer.

2.3 State-of-the-Art Models: The GAPS Architecture

The GAPS architecture is a state-of-the-art model following the Inter-GPS philosophy. Its components are described below.

2.3.1 Problem-Type Classifier

GAPS includes a classifier to identify the type of geometry problem (e.g., calculation or proving) and adapt its solution approach accordingly. This component is **eliminated** for the current purpose, since the task only requires calculating and predicting the right option.

2.3.2 Image Processing

If the input involves diagrams or images, GAPS uses image processing techniques to extract geometric information such as points, lines, and shapes.

2.3.3 Text Processing

For text-based inputs, GAPS employs natural language processing (NLP) to parse and understand the geometric problem description.

2.3.4 Geometry Representation

Converts both image and text inputs into a unified geometric representation that captures key elements and relationships.

2.3.5 Problem Encoding

Encodes the geometric representation into a format suitable for processing by the model, often using vector representations.

2.3.6 Problem Solving

Applies geometric reasoning and rules to solve the encoded problem, generating a step-by-step solution.

2.3.7 Solution Output

Formats the solution into a human-readable form, which may include diagrams, equations, or textual explanations. This component is also **eliminated** as it is not needed for this task.

> **Note:** Pertaining to resource limitations (GPU constraints) and the dataset restriction, the approach applied is inspired by the multiple layers of the GAPS framework, implemented via fine-tuning existing models and test-and-validation-based training of the entire custom model using backpropagation.

3 Model Architecture and Approach

The model architecture implemented is a comprehensive multilayered model design in an attempt to simulate the architecture of GAPS.

3.1 Stage 1: Image Processing Layer

The task of analysing the geometrical images and their labels, and co-relating them with the problem description, is the foundation of decoding the semantics and context of the question. Images in this dataset can be interpreted as consisting of two main parts:

- The geometric figure (arcs, lines, polygons, circles, shaded area, etc.)
- The corresponding text labellings (named angles, vertices, etc.)

Accordingly, the computer vision task is subdivided into two sub-tasks.

3.1.1 Sub-task 1: Geometric Skeleton Understanding via EfficientNet-B4

The first task is to understand the geometric skeleton — the foundational diagram such as polygons, parallel lines intersected by a transversal, and similar figures. This is an apt CNN task for detecting shapes and figures. For this purpose, **EfficientNet-B4** has been fine-tuned using LoRA.

EfficientNet-B4 is a variant within the EfficientNet family of convolutional neural networks, optimised for image classification tasks. It uses compound scaling to uniformly adjust network depth, width, and resolution, achieving high accuracy with computational efficiency. EfficientNet-B4 has **19 million parameters**.

A custom **EfficientNetB4 Adapter** has been developed which performs the following operations:

- Defines a wrapper class (`EfficientNetAdapter`) that extracts convolutional features from a pre-trained EfficientNet-B4 and adds adaptive pooling.
- Uses `peft` to apply LoRA to specific layers (`fc1`, `fc2`) in the network's feature extractor, keeping original weights frozen while training low-rank adapters.
- Configures LoRA with **rank = 16**, **alpha = 32**, and **5% dropout** for parameter-efficient adaptation to new tasks.
- The `fc1` and `fc2` layers in the LoRA configuration represent the fully connected (dense) layers in the EfficientNet-B4 architecture, which appear in the final classification head where features extracted by convolutional layers are mapped to class probabilities or task-specific outputs.

3.1.2 Sub-task 2: Label Text Extraction via EasyOCR

The second task is to extract the text embedded in these images — the labellings of angles and vertices. For this purpose, **EasyOCR** is used, an open-source Optical Character Recognition (OCR) library built on PyTorch, designed to extract text from images and scanned documents.

This is achieved by:

- Initialising an EasyOCR Reader to perform OCR on English text, using GPU if `DEVICE` is set to `'cuda'`.
- Processing each image using the `readtext` method, extracting the detected text from the results, and joining them into a single string (`ocr_text`).

3.2 Stage 2: Preprocessing

The following preprocessing operations are applied:

- **Image Preprocessing:** Converts RGBA images to RGB, resizes to 380 × 380 pixels, and converts to PyTorch tensors.
- **Label Encoding:** Converts ground-truth answers (A / B / C / D) to numerical indices (0 / 1 / 2 / 3).

- **BERT Tokenisation:** Processes the combined text with the BERT tokeniser (max 512 tokens, padded) for NLP input.
- **Option Handling:** Attempts numeric conversion of choice values for hybrid loss calculation; skips non-numeric cases (string-based options are sent via `raw_choices` for hybrid loss calculation, explained later in this report).

3.3 Stage 3: Text Processing via BERT

The next step is to process the text — that is, the problem description. This is a sequence-to-sequence natural language processing task. Not only must the text be processed, but all information captured as embeddings must be correlated with the visual features from the visual embeddings and the OCR label text embeddings.

- The problem description is encoded to an embedding space using a text encoder in the form of **BERT**.
- **bert-base-uncased** is fine-tuned via LoRA with **r = 16**, **alpha = 32**. The `query` and `value` layers (with reference to the attention mechanism of the encoder) are specifically trained so the model learns the text even better during training.
- The next step is to fuse these multiple embeddings to understand the wholesome context — this is achieved via a fusion layer.

3.4 Stage 4: Multimodal Fusion Layer (Multi-Head Attention)

The **FusionLayer** merges visual and textual information into a unified representation for multimodal tasks:

- It first projects image features (from the CNN) and text embeddings (OCR text and problem description) into a shared **512-dimensional space** using separate linear layers.
- These aligned features are then processed by **multi-head attention**, where the image features act as *queries* to dynamically focus on the most relevant parts of the text (the *keys* and *values*). This attention mechanism creates a context-aware fusion of visual and textual cues.
- Finally, the fused output is projected into a richer **2048-dimensional space**, enhancing its expressiveness for downstream tasks like answering questions that require joint understanding of images and text.

3.5 Stage 5: Reasoning via Phi-1.5 (Lightweight LLM)

Phi-1.5 is a **1.3 billion parameter** Transformer-based language model developed by Microsoft, designed for tasks requiring natural language understanding, reasoning, and code generation. It builds on its predecessor, Phi-1, by leveraging high-quality synthetic "textbook-like" data instead of traditional web-crawled data, enhancing safety and reducing biases. Despite its relatively small size, Phi-1.5 achieves performance comparable to models five times larger on benchmarks like common sense reasoning and logical tasks.

Its efficiency, versatility, and open-source nature make it ideal for this task. Specifically, considering the multimodal nature and complexity of the problem, the major constraint is

optimisation of resources (at most a T4 GPU in Colab) — thus a lightweight model with high accuracy was the most suitable choice. Additionally, Phi-1.5 has been trained predominantly on textbook data (science, mathematics, etc.), making it ideal for elementary geometry problems.

The model is fine-tuned via LoRA, which trains only specific layers while freezing the rest of the model. The layers being trained are:

1. **q_proj:** The query projection layer in the transformer's attention mechanism.
2. **k_proj:** The key projection layer in the attention mechanism.

These layers are part of the self-attention blocks and are modified via low-rank matrices during fine-tuning. All other layers (e.g., `v_proj`, MLP layers, embeddings) remain frozen to retain pretrained knowledge.

Why these layers?

- **Attention Focus:** The `q_proj` and `k_proj` layers determine how the model attends to different parts of the input. Adapting them allows task-specific focus without overfitting.
- **Parameter Efficiency:** Training just these layers reduces trainable parameters by approximately 99% compared to full fine-tuning, enabling GPU-friendly training on T4 GPUs.

3.6 Stage 6: T5-Small Decoder

The combined embeddings obtained from the multi-head fusion layer need to be decoded into text in order to be fed as a text-based prompt to the LLM, upon which it can make the prediction. **T5-small decoder** has been utilised for this purpose. During training, it is trained via **teacher forcing**.

The decoder receives the ground-truth target sequence shifted right (prepended with a start token) as `decoder_input_ids`, while the original sequence (with an EOS token) serves as labels. This forces the model to predict the next token based on the true prior tokens — not its own outputs — improving gradient stability. Cross-entropy loss is computed between decoder predictions and actual labels. This method enables efficient training for text-to-text tasks by aligning inputs with expected outputs.

4 Full-Stack Training via Backpropagation

4.1 Data Preparation and Feature Extraction

The training process starts by preparing the data for both visual and textual modalities. Images are preprocessed to ensure they are in RGB format (grayscale images are converted by repeating channels). Text inputs, including problem descriptions and options, are tokenised into input IDs and attention masks using a text encoder (BERT). These preprocessed inputs are then passed through separate encoders: an image encoder extracts features from the images, which are flattened into a fixed-dimensional vector, while the text encoder generates embeddings for the textual inputs. The extracted features are used as the foundation for multimodal reasoning.

4.2 Fusion of Visual and Textual Features

The extracted image and text features are projected into a shared 512-dimensional space using linear layers. These aligned features are then fused using a fusion layer, which employs multi-head attention to combine visual and textual contexts dynamically. The fused representation is expanded to include a sequence dimension (`unsqueeze(1)`) and serves as input to the decoder (Phi-1.5). This fusion ensures that the model can reason jointly about both modalities, enabling it to understand complex relationships between images and text.

4.3 Decoder Input and Forward Pass

The decoder is set up using teacher forcing, where ground-truth labels (answer indices) are prepended with a start token (`decoder_input_ids`). The fused multimodal representation is passed to the decoder, which predicts logits for the possible answer options (A / B / C / D). The forward pass computes these logits by attending to both visual and textual contexts, enabling the model to generate predictions based on the combined information. This step is crucial for tasks like answering questions that require reasoning across both modalities.

4.4 Loss Calculation and Hybrid Loss Design

The training uses two types of losses:

- **Cross-Entropy Loss (CE):** Measures how well the model predicts the correct answer index out of the 4 options.
- **Hybrid Loss:**
  - If numeric values for options are available, **Mean Squared Error (MSE)** compares predicted values with ground truth.
  - If only text options are available, **Cosine Embedding Loss** calculates similarity between predicted embeddings and the correct embeddings generated by the text encoder.

The final loss is a weighted combination of CE loss (**40%**) and hybrid loss (**60%**), balancing classification accuracy with task-specific requirements.

4.5 Optimisation and Validation

The training process uses **mixed precision training** with `autocast` to reduce memory usage and improve efficiency on GPUs. Gradients are scaled using `GradScaler` to prevent underflow during backpropagation.

After each epoch, the model is evaluated on validation data, tracking accuracy as the primary metric (correct predictions divided by total predictions). If validation accuracy improves, the model's state is saved as a checkpoint (`best_model.pth`). This ensures that the best-performing version of the model is retained for deployment or further fine-tuning.

By freezing certain components like encoders and only training specific layers (e.g., the fusion layer), this approach achieves efficient fine-tuning without overfitting or excessive resource usage. The code effectively combines visual and textual reasoning while optimising performance through parameter-efficient techniques like mixed precision training and selective fine-tuning.

5 Hyperparameter Tuning and Results

The accuracy metric utilised for performance evaluation is accuracy — the correctly predicted true positive and true negative values versus the total values predicted.

- **Maximum test accuracy achieved:** 29.83%
- **Maximum validation accuracy achieved:** 33.42%

For context, the state-of-the-art GAPS model reaches approximately 68% accuracy, while other models average around 45%. **100% of the training dataset has been utilised.**

The hyperparameters tuned during training are as follows:

- **LoRA rank:** For Phi-1.5, rank = 8. For BERT and EfficientNet-B4, rank = 16.
- **Learning rates** (determined via hyperparameter tuning):
  - Phi-1.5 model parameters: `lr = 1e-5`
  - Image encoder (EfficientNet-B4): `lr = 1e-5`
  - Text encoder (BERT): `lr = 2e-5`
  - Fusion layer: `lr = 3e-5`
- **Train / validation split:** 70% training, 30% validation.
- **Number of epochs:** 1.
- **Logit normalisation:** Disabled — when enabled, generalisation on unseen data (cross-validation accuracy) reduces.

6 Issues Faced

1. **LoRA target module configuration for EfficientNet-B4 Adapter.** The first issue faced was training via LoRA and setting the `target_modules`, especially for the `EfficientNetB4Adapter`. This was finally resolved by targeting the densely connected layers of the classifier head (`fc1`, `fc2`).

2. **Resource constraints.** One major issue was the GPU resource limitation. Using better reasoning models like **deepseek-math-7b-instruct** was impossible to train on ordinary CPUs and even on the T4 GPU provided by Colab. Settling on a lightweight model was a necessity to reach training completion and to keep VRAM consumption within limits when training on a single T4 GPU.

3. **Backpropagation chain continuity.** Another issue during training was maintaining the continuity of the backpropagation chain — the classic `"0 tensor does not have grad_fn"` error. Ensuring that every component, including the decoder part (enabled via teacher forcing), did not detach from the backpropagation chain was essential so that, based on the predicted output, all layers fine-tune correctly: better image decoding, better text decoding, better fusion of both, and better reasoning — all learned via training.

7 Potential Improvements

1. **Implementing Inter-GPS with a dynamic knowledge graph.** The most impactful improvement would be to actually implement Inter-GPS, most efficiently using an end-to-end graph-based training approach. This would be made even more scalable by building a dynamic knowledge graph — not only consisting of existing theorems but also updated in real time using Retrieval Augmented Generation (RAG), while simultaneously applying a chain-of-thought mechanism.

2. **Custom knowledge graph with chain-of-thought reasoning.** A simpler starting point would be to introduce a custom knowledge graph reinforced with existing mathematical (specifically geometrical) theorems, and induce a step-by-step solving approach via chain of thought to mimic the human way of solving mathematics problems.

3. **Geometry-specific CNN training.** Separate dataset training of the CNN on geometry-specific data only, labelled against geometric primitives (supervised learning), since most CNNs are trained on general natural pictures and are not specific to mathematical diagrams.

4. **Dedicated symbolic analyser.** A symbolic analyser trained exclusively on mathematical symbols to analyse the mathematical expressions involved in the problems.

5. **Upgrading compute resources.** Upgrading to higher GPU resources is an inevitable requirement for achieving better performance, as it would allow training larger models for more epochs.

8 Summary

Table 1: Architecture Component Summary

Component	Role
EfficientNet-B4 (LoRA, r=16)	Extracts convolutional features from geometric diagrams; adaptive pooling and fine-tuned classifier head.
EasyOCR	Extracts text labels (angles, vertices, measurements) embedded in images.
BERT-base-uncased (LoRA, r=16)	Encodes the natural-language problem description into contextual embeddings.
Multi-Head Attention Fusion Layer	Projects image and text features into a shared 512-d space; cross-modal attention; outputs 2048-d fused representation.
T5-Small Decoder (Teacher Forcing)	Decodes the fused embedding into a token sequence suitable as LLM input.
Phi-1.5 (1.3B, LoRA, r=8)	Performs final reasoning and predicts the answer option (A/B/C/D).
Loss function	40% Cross-Entropy + 60% Hybrid (MSE for numeric options; Cosine Embedding Loss for text options).
Test accuracy	29.83%
Validation accuracy	33.42%
Training data used	100%

The MOSAIC-PS2 system presents a resource-efficient multimodal architecture that mirrors the layered philosophy of GAPS and Inter-GPS under strict single-GPU and dataset-only constraints. The system establishes a solid foundation whose accuracy can be substantially improved with additional compute, geometry-specific pre-training, and knowledge graph integration.